

Trabalho 1 - Internet das Coisas - Implementação de um Protocolo de Comunicação Serial

Arthur Bogacki Veríssimo¹, Diego Ribeiro Chaves¹,
Gabriel Souza Baggio¹

¹Centro de Tecnologia - Universidade Federal de Santa Maria - RS, Brasil

1. Introdução

O objetivo do trabalho é desenvolver um protocolo próprio de comunicação serial entre dois Arduinos, capaz de transmitir diferentes tipos de dados, detectar falhas na comunicação e garantir que as mensagens sejam corretamente recebidas. O sistema é composto por dois dispositivos com papéis fixos:

- **Transmissor** (`Transmissor.cpp`): recebe comandos do usuário pelo Monitor Serial, monta as mensagens, envia-as ao receptor e informa o resultado de cada transmissão;
- **Receptor** (`Receptor.cpp`): valida as mensagens recebidas, responde com confirmação positiva ou negativa e apresenta os dados no seu Monitor Serial.

A comunicação entre as placas utiliza a biblioteca `SoftwareSerial`, que emula uma porta serial em pinos digitais, deixando a porta serial nativa (`Serial`) livre para a interação com o usuário [Arduino 2026]. A Tabela 1 apresenta a ligação física entre as placas, na qual o pino de transmissão de cada Arduino é conectado ao pino de recepção do outro.

Tabela 1. Ligação física entre os Arduinos (`SoftwareSerial` portaSerial(10, 11)).

Arduino transmissor		Arduino receptor
Pino 11 (TX)	→	Pino 10 (RX)
Pino 10 (RX)	←	Pino 11 (TX)
GND	—	GND

O restante do documento está organizado da seguinte forma: a Seção 2 apresenta a visão geral do protocolo; a Seção 3 detalha a estrutura das mensagens; a Seção 4 descreve o funcionamento da comunicação; a Seção 5 explica os mecanismos de confiabilidade; a Seção 6 reúne as decisões de projeto; as Seções 7 e 8 apresentam os testes realizados e os resultados obtidos; e a Seção 9 conclui o trabalho.

2. Visão Geral do Protocolo

O protocolo foi organizado em três camadas, de modo que cada parte do código tenha uma responsabilidade bem definida:

1. **API do protocolo:** as funções `sendByte`, `sendWord`, `sendFloat` e `sendData`, que seguem exatamente os protótipos exigidos no enunciado e convertem o valor recebido em uma sequência de bytes (*payload*);

2. **Camada de enlace:** responsável por montar e validar os quadros, calcular o CRC-8, controlar os números de sequência e executar a retransmissão automática;
3. **Interface com o usuário:** menus no Monitor Serial do transmissor para escolha do tipo de dado, do valor e da falha a ser simulada, e mensagens de diagnóstico em ambos os dispositivos.

Todas as funções de envio retornam `true` somente quando o receptor confirma a mensagem, e `false` quando a entrega não pôde ser confirmada. Os parâmetros de operação do protocolo estão resumidos na Tabela 2.

Tabela 2. Parâmetros de operação do protocolo.

Parâmetro	Valor	Uso	Função
BAUD_RATE	4800 bps (8N1)	TX e RX	Taxa das portas serial e <code>SoftwareSerial</code>
TAMANHO_MAX_DATA	64 bytes	TX e RX	Tamanho máximo do <i>payload</i> de dados variáveis
MAX_TENTATIVAS	3	TX	Número máximo de envios de uma mesma mensagem
TIMEOUT_RESPOSTA_MS	500 ms	TX	Tempo máximo de espera por ACK ou NACK
ESPERA_RETRANSMISSAO_MS	200 ms	TX	Intervalo antes de uma retransmissão
TIMEOUT_ENTRE_BYTES_MS	50 ms	RX	Intervalo máximo entre bytes de um quadro
TIMEOUT_LINHA_MS	100 ms	TX	Silêncio que encerra uma entrada digitada

3. Estrutura das Mensagens

3.1. Quadro de dados

Toda mensagem enviada pelo transmissor, independentemente do tipo de dado, utiliza o mesmo formato de quadro, ilustrado na Tabela 3 e detalhado na Tabela 4. Os campos com mais de um byte são transmitidos em ordem *big-endian* (byte mais significativo primeiro).

Tabela 3. Disposição dos campos do quadro de dados (N = tamanho do *payload*).

0xAA	TIPO	SEQ	TAM_H	TAM_L	PAYLOAD	CRC-8
1 byte	1 byte	1 byte	1 byte	1 byte	N bytes	1 byte

3.2. Tipos de dados

O campo Tipo permite que o receptor identifique como interpretar o *payload*. Cada tipo possui um tamanho esperado, verificado pelo receptor antes da leitura dos dados, conforme a Tabela 5.

Tabela 4. Descrição dos campos do quadro de dados.

Posição	Campo	Tamanho	Conteúdo
0	Início	1 byte	Valor fixo 0xAA, que marca o início de um quadro
1	Tipo	1 byte	Identificador do tipo de dado (Tabela 5)
2	Sequência	1 byte	Número da mensagem (0 a 255), incrementado a cada nova mensagem e mantido nas retransmissões
3–4	Tamanho	2 bytes	Quantidade N de bytes do <i>payload</i>
5 a $4 + N$	<i>Payload</i>	N bytes	Dado codificado conforme o tipo
$5 + N$	CRC-8	1 byte	Código de verificação calculado sobre os campos Tipo, Sequência, Tamanho e <i>Payload</i>

Tabela 5. Tipos de dados suportados e codificação do *payload*.

Código	Tipo	Função da API	N	Codificação
0x01	<i>byte</i>	<code>sendByte(uint8_t)</code>	1	O próprio valor
0x02	<i>word</i>	<code>sendWord(uint16_t)</code>	2	Byte alto seguido do byte baixo
0x03	<i>float</i>	<code>sendFloat(float)</code>	4	Os 32 bits do padrão IEEE 754, em <i>big-endian</i>
0x04	<i>dados</i>	<code>sendData(const uint8_t*, uint16_t)</code>	1–64	Os bytes na ordem original

3.3. Quadro de resposta

Para cada quadro recebido, o receptor responde com um quadro de dois bytes (Tabela 6). Os códigos escolhidos correspondem aos caracteres de controle ACK e NAK da tabela ASCII. O número de sequência na resposta permite ao transmissor verificar que a confirmação se refere à mensagem atual.

Tabela 6. Quadro de resposta do receptor.

Campo	Tamanho	Conteúdo
Código	1 byte	0x06 (ACK): quadro válido; 0x15 (NAK): quadro inválido
Sequência	1 byte	Número de sequência do quadro ao qual a resposta se refere

3.4. Exemplos de quadros

A Tabela 7 mostra os bytes efetivamente transmitidos para uma chamada de cada função, com números de sequência ilustrativos. No caso do *float*, o valor 3,1415 corresponde à representação IEEE 754 0x40490E56.

Tabela 7. Exemplos de quadros transmitidos (valores em hexadecimal).

Chamada	SEQ	Quadro transmitido	Resposta (ACK)
<code>sendByte(150)</code>	04	AA 01 04 00 01 96 C4	06 04
<code>sendWord(1024)</code>	05	AA 02 05 00 02 04 00 3D	06 05
<code>sendFloat(3.1415)</code>	06	AA 03 06 00 04 40 49 0E 56 22	06 06
<code>sendData("IoT", 3)</code>	07	AA 04 07 00 03 49 6F 54 B1	06 07

3.5. Sobrecarga e tempo de transmissão

Cada quadro possui 6 bytes de controle além do *payload*. Na configuração 8N1, cada byte ocupa 10 bits na linha (início, 8 bits de dados e parada); portanto, a 4800 bps, cada byte leva aproximadamente 2,08 ms para ser transmitido. A Tabela 8 apresenta o tamanho dos quadros e os tempos teóricos de transmissão.

Tabela 8. Tamanho dos quadros e tempo teórico de transmissão a 4800 bps.

Tipo	Payload	Quadro	Tempo do quadro	Quadro + resposta
<i>byte</i>	1 byte	7 bytes	14,6 ms	18,8 ms
<i>word</i>	2 bytes	8 bytes	16,7 ms	20,8 ms
<i>float</i>	4 bytes	10 bytes	20,8 ms	25,0 ms
dados	1–64 bytes	7–70 bytes	14,6–145,8 ms	18,8–150,0 ms

4. Funcionamento da Comunicação

4.1. Transmissor

O transmissor exibe um menu no Monitor Serial com as opções de envio de *byte*, *word*, *float* e dados de tamanho variável. Para os tipos numéricos, o usuário digita o valor, que é validado quanto ao formato e ao intervalo permitido. Para dados de tamanho variável, o usuário informa primeiro o tamanho (1 a 64 bytes) e depois a mensagem; mensagens menores que o tamanho informado são recusadas, e mensagens maiores são truncadas com aviso. Em seguida, o usuário escolhe se deseja simular alguma falha (Seção 7). A leitura das entradas funciona com ou sem terminador de linha configurado no Monitor Serial.

Após a coleta dos dados, a transmissão segue os passos abaixo:

1. A função da API converte o valor em *payload* e solicita o envio à camada de enlace, que incrementa o número de sequência;
2. Antes de cada tentativa, os bytes pendentes na porta serial são descartados, eliminando respostas atrasadas de tentativas anteriores;
3. O transmissor aguarda o envio das mensagens pendentes ao Monitor Serial (`Serial.flush()`); em seguida, o quadro é montado, o CRC-8 é calculado e todos os bytes são transmitidos;
4. O transmissor aguarda os dois bytes da resposta por até 500 ms;

5. Se receber um ACK com o número de sequência atual, a mensagem é considerada entregue e a função retorna `true`;
6. Se receber um NACK, uma resposta inválida ou nenhuma resposta, aguarda 200 ms e retransmite o mesmo quadro, com o mesmo número de sequência;
7. Após três tentativas sem confirmação, a função retorna `false`.

Ao final, o resultado é apresentado ao usuário como `[RESULTADO] SUCESSO` ou `[RESULTADO] FALHA`, precedido pelo registro de cada tentativa e da resposta obtida.

4.2. Receptor

O receptor monitora continuamente a porta serial. Bytes que não correspondem ao início de um quadro são ignorados, o que permite ressincronizar a comunicação após ruídos ou quadros incompletos. Ao encontrar o byte `0xAA`, o receptor lê o quadro byte a byte, aguardando no máximo 50 ms pela chegada de cada byte, e aplica as verificações da Tabela 9 na ordem apresentada.

Tabela 9. Verificações realizadas pelo receptor para cada quadro.

Etapa	Verificação	Ação em caso de falha
1	Byte lido é o início de quadro (<code>0xAA</code>)	Byte ignorado
2	Cabeçalho (tipo, sequência e tamanho) chega completo	NACK: “quadro incompleto”
3	Tipo conhecido e tamanho compatível com o tipo	NACK: “tipo desconhecido ou tamanho incompatível”
4	<i>Payload</i> e CRC chegam completos	NACK: “quadro incompleto”
5	CRC-8 calculado igual ao CRC-8 recebido	NACK: “CRC inválido”
6	Número de sequência diferente da última mensagem aceita	ACK reenviado e dados descartados (duplicata)

A verificação do tipo e do tamanho ocorre antes da leitura do *payload*, o que impede que um campo de tamanho corrompido provoque a escrita além do *buffer* de 64 bytes. Antes de enviar um NACK, o receptor descarta os bytes restantes do quadro inválido até que a linha permaneça 50 ms em silêncio, evitando que esses bytes sejam interpretados como um novo quadro.

Quando todas as verificações são satisfeitas, o receptor envia o ACK *antes* de apresentar os dados no Monitor Serial. Essa ordem é importante porque a escrita de uma mensagem longa no Monitor Serial a 4800 bps pode bloquear o programa por dezenas de milissegundos, tempo que seria descontado do *timeout* do transmissor. Os dados são apresentados de acordo com o tipo identificado, como nos exemplos:

```
[RX] seq 104 | BYTE: 150
[RX] seq 105 | WORD: 1024
[RX] seq 117 | FLOAT: 3.1415
[RX] seq 107 | DATA (3 bytes): IoT
```

5. Mecanismo de Confiabilidade

5.1. Detecção de erros com CRC-8

A integridade de cada quadro é verificada por um CRC de 8 bits com polinômio gerador $x^8 + x^2 + x + 1$ (0x07), valor inicial zero e sem reflexão de bits, variante conhecida como CRC-8/SMBUS. O cálculo abrange todos os campos do quadro exceto o byte de início, e é refeito pelo receptor e comparado ao valor recebido.

A Tabela 10 compara o CRC-8 com um *checksum* por XOR de todos os bytes, alternativa de mesmo custo (1 byte) avaliada no início do projeto. Os erros de dois bits foram avaliados por enumeração exaustiva de todos os padrões possíveis em cada tamanho de quadro; as trocas, por enumeração de todos os pares de bytes adjacentes distintos em quadros de exemplo. Ambos os métodos detectam todos os erros com número ímpar de bits e todas as rajadas de até 8 bits, e, com corrupção aleatória intensa, ambos deixam passar cerca de 1 em cada 256 quadros. A diferença está nos erros de padrão par, como dois bits invertidos na mesma posição de bytes diferentes, que o XOR não detecta.

Tabela 10. Erros não detectados: CRC-8 versus *checksum* XOR.

Quadro	Erros de 2 bits		Troca de bytes adjacentes	
	CRC-8	XOR	CRC-8	XOR
byte ($N = 1$)	0%	10,6%	0 de 4	4 de 4
word ($N = 2$)	0%	10,9%	0 de 5	5 de 5
float ($N = 4$)	0%	11,3%	0 de 7	7 de 7
dados ($N = 64$)	0,62%	12,3%	1 de 67	67 de 67

O CRC-8 utilizado detecta todos os erros de até três bits enquanto os dados verificados não ultrapassam 119 bits (cabeçalho de 4 bytes e *payload* de até 10 bytes), resultado coerente com a análise de polinômios de CRC para redes embarcadas apresentada em [Koopman and Chakravarty 2004]. Isso cobre integralmente os tipos *byte*, *word* e *float*. Para mensagens maiores essa garantia deixa de ser absoluta, mas a taxa de erros não detectados permanece cerca de 20 vezes menor que a do XOR.

5.2. Retransmissão automática (*Stop-and-Wait* ARQ)

A recuperação de falhas utiliza o esquema de repetição automática *Stop-and-Wait* [Tanenbaum and Wetherall 2011]: o transmissor envia um quadro e aguarda sua confirmação antes de enviar o próximo. A retransmissão é disparada por três eventos: recebimento de NACK, que indica que o receptor detectou um erro e solicita nova transmissão; *timeout*, quando nenhuma resposta chega em 500 ms; e resposta inválida, quando o código ou o número de sequência não conferem. O limite de três tentativas garante que o transmissor não fique bloqueado indefinidamente quando o receptor está ausente. A Tabela 11 resume como cada tipo de falha é detectado e tratado.

5.3. Números de sequência e mensagens duplicadas

Se o quadro de dados chega corretamente, mas o ACK se perde, o transmissor retransmite uma mensagem que o receptor já processou. Sem tratamento, o dado seria apresentado

Tabela 11. Falhas de comunicação e reação do protocolo.

Falha	Detecção	Reação
Bits corrompidos no quadro	CRC-8 divergente ou cabeçalho inválido	NACK e retransmissão
Byte perdido no meio do quadro	Intervalo entre bytes excedido, cabeçalho inválido ou CRC	NACK e retransmissão
Quadro inteiro perdido ou byte de início corrompido	Ausência de resposta em 500 ms	Retransmissão por <i>timeout</i>
ACK ou NACK perdido	Ausência de resposta em 500 ms	Retransmissão; receptor descarta a duplicata
ACK corrompido	Código ou sequência não conferem	Retransmissão; receptor descarta a duplicata
Resposta atrasada de tentativa anterior	—	Descartada antes da nova tentativa
Bytes espúrios na linha	Não formam um quadro válido	Ignorados até o próximo 0xAA
Receptor ausente ou desconectado	Três <i>timeouts</i> consecutivos	Falha informada ao usuário

duas vezes. Para evitar isso, o transmissor atribui um número de sequência a cada nova mensagem e o mantém nas retransmissões. O receptor armazena o número da última mensagem aceita: ao receber um quadro válido com o mesmo número, reenvia o ACK, necessário para liberar o transmissor, mas descarta os dados.

O número de sequência inicial do transmissor é sorteado na inicialização. Assim, caso apenas o transmissor seja reiniciado, a primeira mensagem dificilmente coincidirá com o último número aceito pelo receptor e não será descartada por engano.

5.4. Limitações conhecidas

- O esquema *Stop-and-Wait* envia uma mensagem por vez, o que limita a vazão. Esquemas com janela deslizante, como *Go-Back-N*, aumentariam a vazão ao custo de maior complexidade e uso de memória;
- Se todas as confirmações de uma mensagem se perderem, o transmissor informa falha, embora o receptor tenha recebido os dados. Essa ambiguidade é inerente a confirmações por *timeout*, pois o transmissor não distingue a perda do quadro da perda da resposta;
- O byte 0xAA pode aparecer dentro do *payload*, pois não há inserção de bytes de escape. Em caso de perda de sincronismo, um falso início de quadro é rejeitado pelas verificações de tamanho, *timeout* e CRC;
- O tamanho máximo de dados variáveis é 64 bytes, valor escolhido considerando a memória RAM de 2 KB do Arduino Uno, na qual os *buffers* das mensagens são alocados;
- As constantes do protocolo são duplicadas nos dois programas, pois cada Arduino é programado com um único arquivo, e devem ser mantidas idênticas.

6. Decisões de Projeto

A Tabela 12 reúne as principais decisões tomadas durante o desenvolvimento e suas justificativas.

Tabela 12. Principais decisões de projeto.

Decisão	Justificativa
Formato único de quadro para todos os tipos	Um único procedimento de montagem e validação atende a todos os tipos, simplificando o código e os testes
Campo de tamanho com 2 bytes	Compatível com o parâmetro <code>uint16_t size</code> de <code>sendData</code> ; custa 1 byte adicional nos tipos de tamanho fixo
Tipo e tamanho validados em conjunto	Permite ao receptor identificar corretamente o dado e rejeitar quadros incoerentes antes de ler o <i>payload</i>
Ordem <i>big-endian</i> em todos os campos	Torna a interpretação independente da arquitetura, inclusive para o <i>float</i> , serializado pelos seus 32 bits
CRC-8 no lugar de <i>checksum</i> XOR	Mesmo custo de 1 byte, com detecção muito superior de erros de padrão par e de troca de bytes (Tabela 10)
<i>Stop-and-Wait</i> com ACK e NACK	Simples e com pouco uso de memória; adequado à interação do usuário, que envia uma mensagem por vez. O NACK acelera a recuperação (cerca de 70 ms em quadros curtos, contra 500 ms de <i>timeout</i>)
Número de sequência de 8 bits	Evita duplicatas e associa cada resposta à mensagem atual; também facilita o acompanhamento dos registros
ACK enviado antes da impressão	Impede que a escrita no Monitor Serial consuma o <i>timeout</i> do transmissor
<i>Timeout</i> de resposta de 500 ms	Mais de três vezes o pior caso de quadro e resposta (150 ms), sem tornar a detecção de falhas lenta
<i>Timeout</i> entre bytes de 50 ms	Cerca de 24 vezes o tempo de um byte, tolerando atrasos sem demorar a rejeitar quadros incompletos
Taxa de 4800 bps	Taxa moderada, que dá margem de temporização à porta serial implementada em software
Monitor Serial esvaziado antes de cada transmissão	A <code>SoftwareSerial</code> amostra os bits por temporização; evitar interrupções da porta serial nativa durante a troca de quadros reduz erros na leitura da resposta
Leitura do quadro byte a byte	O quadro máximo (70 bytes) excede o <i>buffer</i> de recepção de 64 bytes da <code>SoftwareSerial</code> [Arduino 2026]; consumir os bytes à medida que chegam evita perdas
Simulação de falhas por variável global	Permite demonstrar falhas sem alterar os protótipos exigidos das funções de envio

7. Testes Realizados

7.1. Simulação de falhas no transmissor

Para demonstrar o comportamento do protocolo diante de falhas, o transmissor oferece, antes de cada envio, as opções da Tabela 13. A corrupção inverte um bit do primeiro byte

do *payload* sem recalcular o CRC, reproduzindo o efeito de ruído na linha. As perdas são simuladas no próprio transmissor, que deixa de enviar o quadro ou ignora a confirmação recebida. A ausência do receptor pode ser demonstrada desconectando o fio entre as placas.

Tabela 13. Opções de simulação de falhas do transmissor.

Opção	Falha simulada	Comportamento esperado
0	Nenhuma	Entrega na primeira tentativa
1	Corromper a primeira tentativa	NACK, retransmissão e entrega
2	Corromper todas as tentativas	Três NACKs e falha definitiva
3	Perder o quadro na primeira tentativa	<i>Timeout</i> , retransmissão e entrega
4	Perder o ACK na primeira tentativa	Retransmissão, duplicata descartada pelo receptor e entrega

7.2. Ambiente e cenários de teste

Os testes foram executados em um ambiente de simulação em C++ construído para o trabalho. Nele, os dois programas (*Transmissor.cpp* e *Receptor.cpp*) são compilados sem alterações, com implementações simplificadas das bibliotecas do Arduino, e executados simultaneamente, conectados por um canal serial emulado. O canal reproduz o tempo de transmissão de cada byte a 4800 bps (cerca de 2,08 ms) e a capacidade dos *buffers* de recepção do Arduino, além de permitir a injeção de falhas físicas que não dependem do menu: desconexão do receptor, perda de um byte e corrupção da resposta. As entradas do Monitor Serial foram roteirizadas, e os programas foram compilados sem avisos com as opções `-Wall` e `-Wextra` do GCC.

Os cenários cobrem os quatro tipos de dados, valores nos limites dos intervalos, todas as opções da Tabela 13, falhas físicas na linha e entradas inválidas do usuário. Os mesmos cenários baseados no menu podem ser reproduzidos no Tinkercad ou nas placas físicas.

8. Resultados Obtidos

A Tabela 14 apresenta os resultados dos cenários de transmissão. O tempo total corresponde ao intervalo entre o início da primeira tentativa e o resultado final no transmissor. Todos os cenários apresentaram o comportamento esperado.

Os tempos medidos são coerentes com a análise teórica. No cenário 1, o valor de 21 ms se aproxima dos 18,8 ms previstos na Tabela 8. No cenário 5, a primeira tentativa é rejeitada após cerca de 72 ms (16,7 ms do quadro, 50 ms de descarte e 4,2 ms do NACK), seguidos de 200 ms de espera e de uma nova transmissão bem-sucedida. No cenário 9, o tempo de 1948 ms corresponde a três esperas de 500 ms somadas aos quadros e aos dois intervalos de 200 ms, próximo dos 1944 ms previstos. Esse é o tempo máximo que o usuário aguarda antes de ser informado de uma falha no envio de um *byte*; para dados de 64 bytes, o limite teórico é de cerca de 2,3 s. A comparação entre os cenários 5 e 7 mostra a vantagem do NACK: a recuperação por NACK foi mais de 400 ms mais rápida que a recuperação por *timeout*.

Tabela 14. Resultados dos cenários de transmissão.

#	Cenário	Comportamento observado	Resultado	Tempo
1	Byte 150, sem falha	ACK na 1. ^a tentativa; RX exibe 150	Sucesso	21 ms
2	Float 3,1415 e -12,5, sem falha	ACK na 1. ^a tentativa; RX exibe 3.1415 e -12.5000	Sucesso	27 ms
3	Byte 170 (0xAA no payload)	ACK na 1. ^a tentativa; RX exibe 170	Sucesso	20 ms
4	Word 65535 (valor máximo)	ACK na 1. ^a tentativa; RX exibe 65535	Sucesso	23 ms
5	Word 1024, corromper a 1. ^a (opção 1)	RX: CRC inválido e NACK; ACK na 2. ^a tentativa	Sucesso	295 ms
6	Float 3,1415, corromper todas (opção 2)	Três NACKs; RX não exibe dados	Falha	631 ms
7	Dados "IoT", perder o quadro (opção 3)	Timeout na 1. ^a ; ACK na 2. ^a tentativa	Sucesso	726 ms
8	Dados com 64 bytes, perder o ACK (opção 4)	ACK ignorado; na 2. ^a tentativa RX reconhece a duplicata e exibe a mensagem uma única vez	Sucesso	517 ms
9	Receptor desconectado	Três timeouts	Falha	1948 ms
10	Byte perdido no meio de um quadro de 10 bytes	RX: tamanho incompatível e NACK; ACK na 2. ^a tentativa	Sucesso	330 ms
11	ACK corrompido na linha (word 42)	Resposta inválida; na 2. ^a tentativa RX reconhece a duplicata	Sucesso	246 ms
12	Bytes espúrios com 0xAA antes de um quadro	RX rejeita o falso quadro; TX descarta o NACK atrasado e entrega na 1. ^a tentativa	Sucesso	23 ms

A seguir, os registros dos Monitores Seriais no cenário 5, que demonstra a recuperação após um quadro corrompido:

Transmissor:

```
[TX] Tentativa 1/3 (seq 105)
[SIMULACAO] Quadro corrompido (1 bit invertido)
[TX] NACK recebido: receptor detectou erro no quadro
[TX] Tentativa 2/3 (seq 105)
[TX] ACK recebido
[RESULTADO] SUCESSO: mensagem confirmada pelo receptor
```

Receptor:

```
[RX] NACK enviado: CRC invalido
[RX] seq 105 | WORD: 1024
```

E no cenário 8, no qual a perda do ACK leva a uma retransmissão que o receptor identifica como duplicata (conteúdo da mensagem abreviado):

```
Transmissor:
[TX] Tentativa 1/3 (seq 108)
[SIMULACAO] ACK perdido na linha (tratado como timeout)
[TX] Tentativa 2/3 (seq 108)
[TX] ACK recebido
[RESULTADO] SUCESSO: mensagem confirmada pelo receptor

Receptor:
[RX] seq 108 | DATA (64 bytes): ABCDEFGHIJKLMNOPQRSTUVWXYZ[...]
[RX] seq 108 | duplicata: ACK reenviado e dados descartados
```

As entradas inválidas também foram tratadas corretamente, conforme a Tabela 15. Em nenhum desses casos houve transmissão de dados incorretos.

Tabela 15. Tratamento de entradas do usuário.

Entrada	Comportamento observado
Opção de menu 9	Mensagem de opção inválida e retorno ao menu
Byte 300	Mensagem de valor inválido; nada é transmitido
Tamanho 5 com a mensagem “IoT”	Mensagem menor que o tamanho informado; nada é transmitido
Tamanho 3 com a mensagem “Hello”	Aviso de truncamento; RX recebe “Hel”
Opção de falha “x”	Nova solicitação até uma opção válida ser digitada
Monitor Serial sem terminador de linha ou com CR+LF	Entradas interpretadas corretamente nos dois casos

9. Conclusão

O protocolo desenvolvido transmite dados dos tipos *byte*, *word*, *float* e de tamanho definido pelo usuário entre dois Arduinos, utilizando um formato de quadro único com identificação de tipo, controle de sequência e verificação por CRC-8. O esquema *Stop-and-Wait* com ACK, NACK, *timeout* e limite de tentativas recuperou a comunicação em todos os cenários de falha recuperáveis e informou corretamente as falhas definitivas. A Tabela 16 relaciona cada requisito do trabalho ao mecanismo que o atende.

Como trabalhos futuros, podem ser avaliados um esquema com janela deslizante para aumentar a vazão, a inserção de bytes de escape para eliminar falsos inícios de quadro e um CRC de 16 bits para ampliar as garantias de detecção em mensagens longas.

Referências

Arduino (2026). SoftwareSerial Library. Disponível em: <https://docs.arduino.cc/learn/built-in-libraries/software-serial/>. Acesso em: set. 2026.

Tabela 16. Atendimento aos requisitos do trabalho.

Requisito	Como é atendido	Seção
Comunicação entre dois Arduinos por porta serial	<code>SoftwareSerial</code> nos pinos 10 e 11 a 4800 bps	2
Transmitir <i>byte</i> , <i>word</i> e <i>float</i>	<code>sendByte</code> , <code>sendWord</code> e <code>sendFloat</code> , com os protótipos exigidos	3
Transmitir dados de tamanho definido pelo usuário	<code>sendData</code> com tamanho de 1 a 64 bytes informado pelo usuário	3
Receptor identificar corretamente os dados	Campo Tipo, validação de tamanho e apresentação por tipo	4
Detectar falhas de comunicação	CRC-8, validação do cabeçalho, <i>timeouts</i> e números de sequência	5
Retransmitir mensagens quando necessário	<i>Stop-and-Wait</i> ARQ com até três tentativas	5
Tratar ausência de resposta do receptor	<i>Timeout</i> de 500 ms por tentativa	5
Informar o resultado ao usuário	Registro de cada tentativa e resultado final de sucesso ou falha	4
Demonstrar o comportamento diante de falhas	Menu de simulação de falhas no transmissor	7

Koopman, P. and Chakravarty, T. (2004). Cyclic redundancy code (CRC) polynomial selection for embedded networks. In *International Conference on Dependable Systems and Networks (DSN)*, pages 145–154. IEEE.

Tanenbaum, A. S. and Wetherall, D. J. (2011). *Computer Networks*. Pearson, 5th edition.